

Problem Decomposition

Learning Objectives

- Break complex problems into manageable sub-problems
- Identify dependencies and relationships between sub-problems
- Apply top-down and bottom-up decomposition strategies

Engage (5-7 minutes)

Show students a complex task: planning a school festival. Ask them to list everything needed. The list will be overwhelming. Then ask: can we group these into categories (venue, food, entertainment, logistics)? Demonstrate that decomposition transforms chaos into manageable components.

Explore (10-12 minutes)

Students tackle a real problem: designing a library management system. Using top-down decomposition, they break it into modules (user management, catalog, borrowing, returns, reporting). For each module, they identify inputs, outputs, and dependencies. Groups present their decomposition and justify their module boundaries.

Explain (10-12 minutes)

Problem decomposition is the technique of breaking a complex problem into smaller, more manageable sub-problems that can be solved independently. Top-down decomposition starts with the big picture and progressively refines it—suitable when the overall structure is understood. Bottom-up decomposition starts with known components and assembles them—useful when building from existing pieces. Effective decomposition identifies: functional boundaries (what each piece does), data flow (how pieces communicate), and dependencies (what must be completed first).

Elaborate (8-10 minutes)

Examine decomposition in professional software engineering: microservices architecture, modular programming, and API design. Discuss the Single Responsibility Principle—each module should do one thing well. Analyze how poor decomposition leads to tight coupling and maintenance nightmares. Practice decomposition on diverse problems: an online shop, a traffic management system, or a hospital appointment system.

Evaluate (5-8 minutes)

Students decompose a provided complex problem into modules, draw a module dependency diagram, and write pseudocode for two sub-problems. Peer review: evaluate whether the decomposition is logical, whether modules are appropriately sized, and whether dependencies are correctly identified.